```java
package Grafos;

import java.util.ArrayList;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Teste {

    public static void main(String[] args) {

        Scanner entrada = new Scanner(System.in);
        Grafo grafo = new Grafo(10);

        Vertice[] vertices = new Vertice[10];
        Aresta[] arestas = new Aresta[20];

        int opcao = -1;
        int quantidadeVertices = 0;
        int quantidadeArestas = 0;

        do {
            System.out.println("\n===== MENU DO GRAFO =====");
            System.out.println("1 - Inserir vertice");
            System.out.println("2 - Inserir aresta");
            System.out.println("3 - Remover vertice");
            System.out.println("4 - Remover aresta");
            System.out.println("5 - Verificar adjacencia");
            System.out.println("6 - Consultar vertice oposto");
            System.out.println("7 - Consultar vertices finais");
            System.out.println("8 - Substituir elemento de vertice");
            System.out.println("9 - Substituir elemento de aresta");
            System.out.println("10 - Verificar se o grafo esta vazio");
            System.out.println("11 - Verificar se aresta e direcionada");
            System.out.println("12 - Listar vertices");
            System.out.println("13 - Listar arestas");
            System.out.println("14 - Grau do vertice");
            System.out.println("15 - Grau de entrada do vertice");
            System.out.println("16 - Grau de saida do vertice");
            System.out.println("17 - Arestas incidentes ao vertice");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opcao: ");

            opcao = -1;

            try {
                opcao = entrada.nextInt();
                entrada.nextLine();

                switch (opcao) {

                    case 1: {
                        if (quantidadeVertices >= vertices.length) {
                            System.out.println("Limite de vertices do teste atingido.");
                            break;
                        }

                        System.out.print("Digite o elemento do vertice: ");
                        String elemento = entrada.nextLine();

                        Vertice v = new Vertice();
                        v.setElement(elemento);

                        grafo.insertVertice(v);
                        vertices[quantidadeVertices] = v;
```

```
65
66                          System.out.println("Vertice inserido com sucesso. Indice: "
67                                  + quantidadeVertices);
68
69                          quantidadeVertices++;
70                          break;
71                      }
72
73                      case 2: {
74                          if (quantidadeArestas >= arestas.length) {
75                              System.out.println("Limite de arestas do teste atingido.");
76                              break;
77                          }
78
79                          System.out.print("Indice do vertice de origem: ");
80                          int origem = entrada.nextInt();
81
82                          System.out.print("Indice do vertice de destino: ");
83                          int destino = entrada.nextInt();
84                          entrada.nextLine();
85
86                          if (origem < 0 || origem >= vertices.length
87                                  || destino < 0 || destino >= vertices.length
88                                  || vertices[origem] == null
89                                  || vertices[destino] == null) {
90                              System.out.println("Indice invalido ou vertice inexistente.");
91                              break;
92                          }
93
94                          System.out.print("Elemento da aresta: ");
95                          String elemento = entrada.nextLine();
96
97                          System.out.print("Aresta direcionada? (s/n): ");
98                          String resposta = entrada.nextLine();
99
100                         Aresta a = new Aresta();
101                         a.setElement(elemento);
102
103                         if (resposta.equalsIgnoreCase("s")) {
104                             grafo.insertArestaDirecionada(a, vertices[origem], vertices[destino]);
105                         } else {
106                             grafo.insertAresta(a, vertices[origem], vertices[destino]);
107                         }
108
109                         arestas[quantidadeArestas] = a;
110
111                         System.out.println("Aresta inserida com sucesso. Indice: "
112                                 + quantidadeArestas);
113
114                         quantidadeArestas++;
115                         break;
116                     }
117
118                     case 3: {
119                         System.out.print("Indice do vertice a remover: ");
120                         int indice = entrada.nextInt();
121                         entrada.nextLine();
122
123                         if (indice < 0 || indice >= vertices.length
124                                 || vertices[indice] == null) {
125                             System.out.println("Vertice inexistente.");
126                             break;
127                         }
128
129                         Vertice removido = vertices[indice];
130                         grafo.removeVertice(removido);
```

```
131                         vertices[indice] = null;
132
133                         // o grafo apagou as arestas ligadas a esse vertice,
134                         // entao o teste tambem esquece essas arestas
135                         for (int k = 0; k < arestas.length; k++) {
136                             if (arestas[k] != null
137                                     && (arestas[k].getOrigem() == removido
138                                     || arestas[k].getFim() == removido)) {
139                                 arestas[k] = null;
140                             }
141                         }
142
143                         System.out.println("Vertice removido com sucesso.");
144                         break;
145                     }
146
147                     case 4: {
148                         System.out.print("Indice da aresta a remover: ");
149                         int indice = entrada.nextInt();
150                         entrada.nextLine();
151
152                         if (indice < 0 || indice >= arestas.length
153                                 || arestas[indice] == null) {
154                             System.out.println("Aresta inexistente.");
155                             break;
156                         }
157
158                         grafo.removeAresta(arestas[indice]);
159                         arestas[indice] = null;
160
161                         System.out.println("Aresta removida com sucesso.");
162                         break;
163                     }
164
165                     case 5: {
166                         System.out.print("Indice do primeiro vertice: ");
167                         int i = entrada.nextInt();
168
169                         System.out.print("Indice do segundo vertice: ");
170                         int j = entrada.nextInt();
171
172                         System.out.print("Indice da aresta: ");
173                         int k = entrada.nextInt();
174                         entrada.nextLine();
175
176                         if (i < 0 || i >= vertices.length
177                                 || j < 0 || j >= vertices.length
178                                 || k < 0 || k >= arestas.length
179                                 || vertices[i] == null
180                                 || vertices[j] == null
181                                 || arestas[k] == null) {
182                             System.out.println("Vertice ou aresta inexistente.");
183                             break;
184                         }
185
186                         System.out.println("Sao adjacentes: "
187                                 + grafo.isAdjacent(vertices[i], vertices[j], arestas[k]));
188                         break;
189                     }
190
191                     case 6: {
192                         System.out.print("Indice do vertice: ");
193                         int i = entrada.nextInt();
194
195                         System.out.print("Indice da aresta: ");
196                         int j = entrada.nextInt();
```

```
197                         entrada.nextLine();
198
199                         if (i < 0 || i >= vertices.length
200                                 || j < 0 || j >= arestas.length
201                                 || vertices[i] == null || arestas[j] == null) {
202                             System.out.println("Vertice ou aresta inexistente.");
203                             break;
204                         }
205
206                         Vertice oposto = grafo.oposto(vertices[i], arestas[j]);
207                         System.out.println("Vertice oposto: " + oposto.getElement());
208                         break;
209                     }
210
211                     case 7: {
212                         System.out.print("Indice da aresta: ");
213                         int i = entrada.nextInt();
214                         entrada.nextLine();
215
216                         if (i < 0 || i >= arestas.length || arestas[i] == null) {
217                             System.out.println("Aresta inexistente.");
218                             break;
219                         }
220
221                         Vertice[] finais = grafo.finalVertices(arestas[i]);
222
223                         System.out.println("Vertice de origem: "
224                                 + finais[0].getElement());
225                         System.out.println("Vertice de destino: "
226                                 + finais[1].getElement());
227                         break;
228                     }
229
230                     case 8: {
231                         System.out.print("Indice do vertice: ");
232                         int i = entrada.nextInt();
233                         entrada.nextLine();
234
235                         if (i < 0 || i >= vertices.length || vertices[i] == null) {
236                             System.out.println("Vertice inexistente.");
237                             break;
238                         }
239
240                         System.out.print("Novo elemento: ");
241                         String elemento = entrada.nextLine();
242
243                         grafo.substituirVertice(vertices[i], elemento);
244                         System.out.println("Vertice atualizado.");
245                         break;
246                     }
247
248                     case 9: {
249                         System.out.print("Indice da aresta: ");
250                         int i = entrada.nextInt();
251                         entrada.nextLine();
252
253                         if (i < 0 || i >= arestas.length || arestas[i] == null) {
254                             System.out.println("Aresta inexistente.");
255                             break;
256                         }
257
258                         System.out.print("Novo elemento: ");
259                         String elemento = entrada.nextLine();
260
261                         grafo.substituirAresta(arestas[i], elemento);
262                         System.out.println("Aresta atualizada.");
```

```
263                         break;
264                     }
265
266                     case 10:
267                         System.out.println("Grafo vazio: " + grafo.isEmpty());
268                         break;
269
270                     case 11: {
271                         System.out.print("Indice da aresta: ");
272                         int i = entrada.nextInt();
273                         entrada.nextLine();
274
275                         if (i < 0 || i >= arestas.length || arestas[i] == null) {
276                             System.out.println("Aresta inexistente.");
277                             break;
278                         }
279
280                         System.out.println("Direcionada: " + grafo.isDirected(arestas[i]));
281                         break;
282                     }
283
284                     case 12: {
285                         Vertice[] lista = grafo.vertices();
286
287                         for (int i = 0; i < lista.length; i++) {
288                             if (lista[i] != null) {
289
290                                 // descobre o indice do vertice no teste
291                                 int indice = -1;
292                                 for (int k = 0; k < vertices.length; k++) {
293                                     if (vertices[k] == lista[i]) {
294                                         indice = k;
295                                     }
296                                 }
297
298                                 System.out.println("Vertice " + indice + ": "
299                                         + lista[i].getElement());
300                             }
301                         }
302                         break;
303                     }
304
305                     case 13: {
306                         ArrayList<Aresta>[][] matriz = grafo.arestas();
307                         ArrayList<Aresta> vistas = new ArrayList<Aresta>();
308
309                         for (int i = 0; i < matriz.length; i++) {
310                             for (int j = 0; j < matriz[i].length; j++) {
311                                 for (Aresta a : matriz[i][j]) {
312                                     if (!vistas.contains(a)) {
313                                         vistas.add(a);
314
315                                         // descobre o indice da aresta no teste
316                                         int indice = -1;
317                                         for (int k = 0; k < arestas.length; k++) {
318                                             if (arestas[k] == a) {
319                                                 indice = k;
320                                             }
321                                         }
322
323                                         System.out.println("Aresta " + indice
324                                                 + " [" + a.getElement() + "]: "
325                                                 + a.getOrigem().getElement()
326                                                 + (a.getDirecionada() ? " -> " : " -- ")
327                                                 + a.getFim().getElement());
328                                     }
```

```
                            }
                        }
                    }

                    if (vistas.isEmpty()) {
                        System.out.println("Nao ha arestas.");
                    }
                    break;
                }

                case 14: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length || vertices[i] == null) {
                        System.out.println("Vertice inexistente.");
                        break;
                    }

                    System.out.println("Grau: " + grafo.grau(vertices[i]));
                    break;
                }

                case 15: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length || vertices[i] == null) {
                        System.out.println("Vertice inexistente.");
                        break;
                    }

                    System.out.println("Grau de entrada: " + grafo.grauEntrada(vertices[i]));
                    break;
                }

                case 16: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length || vertices[i] == null) {
                        System.out.println("Vertice inexistente.");
                        break;
                    }

                    System.out.println("Grau de saida: " + grafo.grauSaida(vertices[i]));
                    break;
                }

                case 17: {
                    System.out.print("Indice do vertice: ");
                    int i = entrada.nextInt();
                    entrada.nextLine();

                    if (i < 0 || i >= vertices.length || vertices[i] == null) {
                        System.out.println("Vertice inexistente.");
                        break;
                    }

                    ArrayList<Aresta> incidentes = grafo.arestasIncidentes(vertices[i]);

                    if (incidentes.isEmpty()) {
                        System.out.println("Nenhuma aresta incidente.");
```

```
395                         }
396
397                         for (Aresta a : incidentes) {
398
399                             // descobre o indice da aresta no teste
400                             int indice = -1;
401                             for (int k = 0; k < arestas.length; k++) {
402                                 if (arestas[k] == a) {
403                                     indice = k;
404                                 }
405                             }
406
407                             System.out.println("Aresta " + indice + " [" + a.getElement() + "]");
408                         }
409                         break;
410                     }
411
412                     case 0:
413                         System.out.println("Encerrando o programa...");
414                         break;
415
416                     default:
417                         System.out.println("Opcao invalida.");
418                 }
419
420             } catch (Correcao e) {
421                 System.out.println("Erro: " + e.getMessage());
422
423             } catch (InputMismatchException e) {
424                 // usuario digitou texto onde era numero
425                 entrada.nextLine();
426                 System.out.println("Entrada invalida. Digite um numero.");
427             }
428
429         } while (opcao != 0);
430
431         entrada.close();
432     }
433 }
```